FPT Education
FPT UNIVERSITY

Chapter 7: Bootloaders

U-Boot • Boot Flow • Device Tree • Embedded Boot

FPT Education
FPT UNIVERSITY

Learning Objectives

- Understand role of bootloader
- Analyze boot challenges in embedded systems
- Learn U-Boot basics
- Understand Device Tree concept
- Understand Porting U-Boot

FPT Education
FPT UNIVERSITY

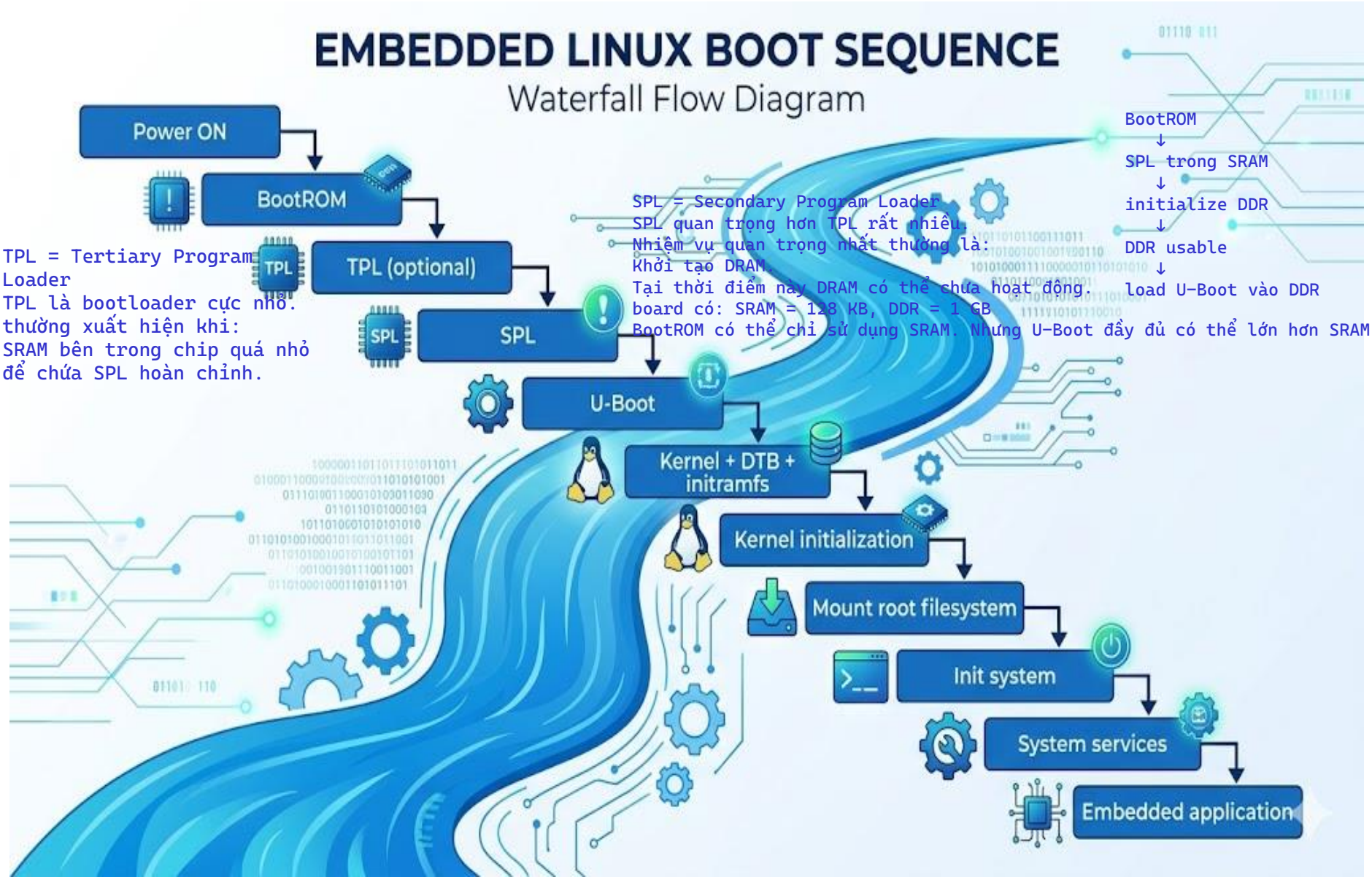
What is a Bootloader?

- First code executed after reset
- Initializes CPU, clocks, DRAM
- Loads kernel, DTB, initramfs

 Bootloader bridges hardware-specific startup and OS-independent kernel execution

FPT Education

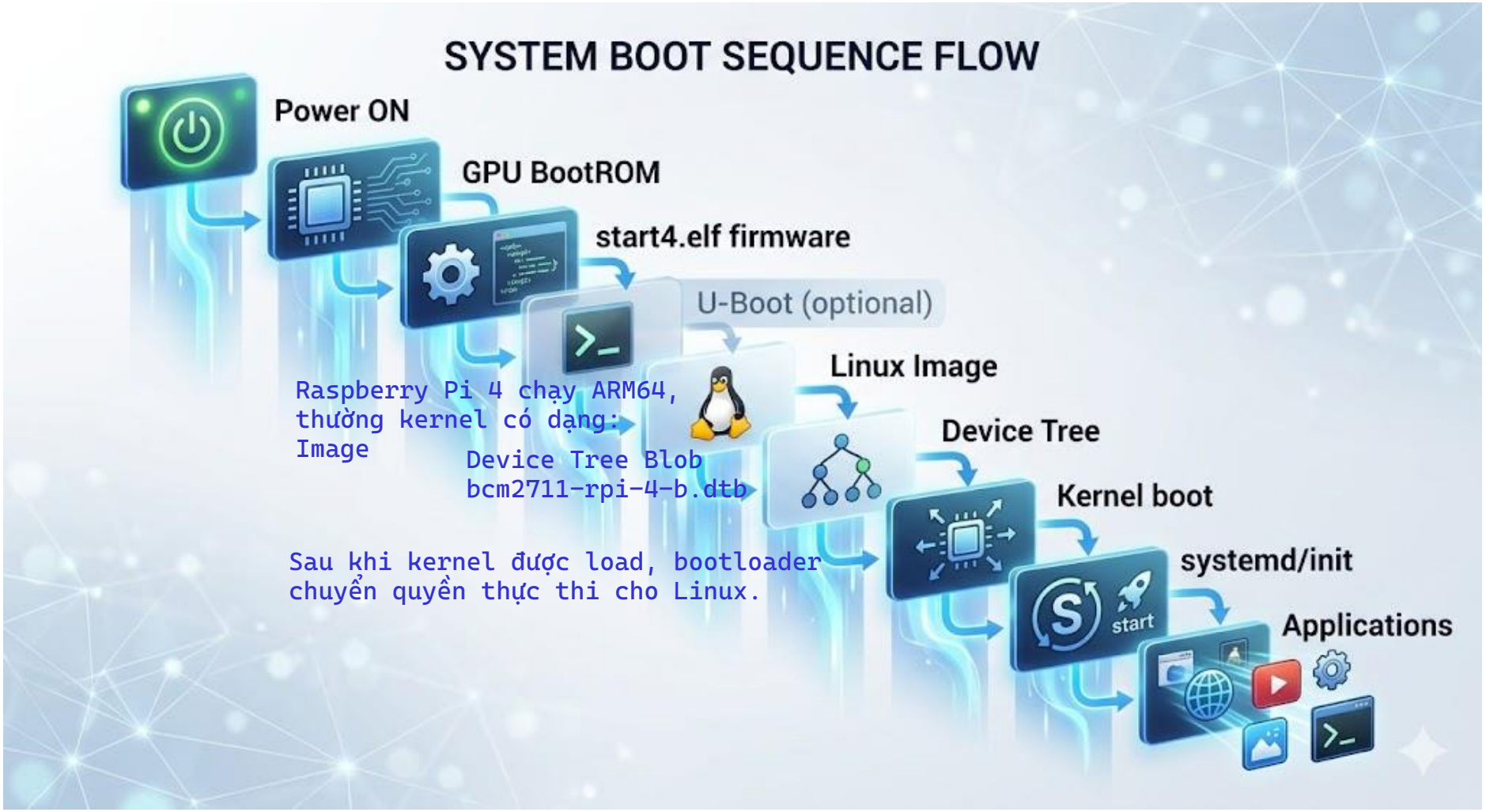
FPT UNIVERSITY



Thành phần	Chạy ở đâu	Mục đích
BootROM	Internal ROM	Tìm bootloader đầu tiên
TPL	SRAM	Chuẩn bị cho SPL
SPL	SRAM	Init DDR, load U-Boot
U-Boot	DRAM	Load Linux

FPT Education
FPT UNIVERSITY

Example: Raspberry Pi 4 Boot Flow



Raspberry Pi không bắt buộc phải có U-Boot.
Cách boot thông thường:

```
Firmware
   ↓
Linux kernel
```

Nhưng khi học Embedded Linux, ta thường chèn U-Boot vào:

```
Firmware
   ↓
U-Boot
   ↓
Linux kernel
```

U-Boot là một bootloader mạnh, cho phép:
chọn kernel;
thay đổi kernel command line;
load kernel từ SD, USB, network;
load Device Tree;
chạy script boot;
debug quá trình boot;
boot nhiều hệ điều hành khác nhau.
Ví dụ từ U-Boot ta có thể dùng các lệnh:
printenv
setenv
mmc info
fatls
fatload
booti

```
Device Tree
uart0: serial@7e201000 {
    compatible = "arm,pl011";
    interrupts = < ... >;
    status = "okay";
};
```

Tầng	Thành phần
1. Hardware boot	Boot ROM + EEPROM
2. Bootloader/Firmware	`start4.elf`, U-Boot
3. Kernel	Linux `Image`
4. User space	systemd + applications

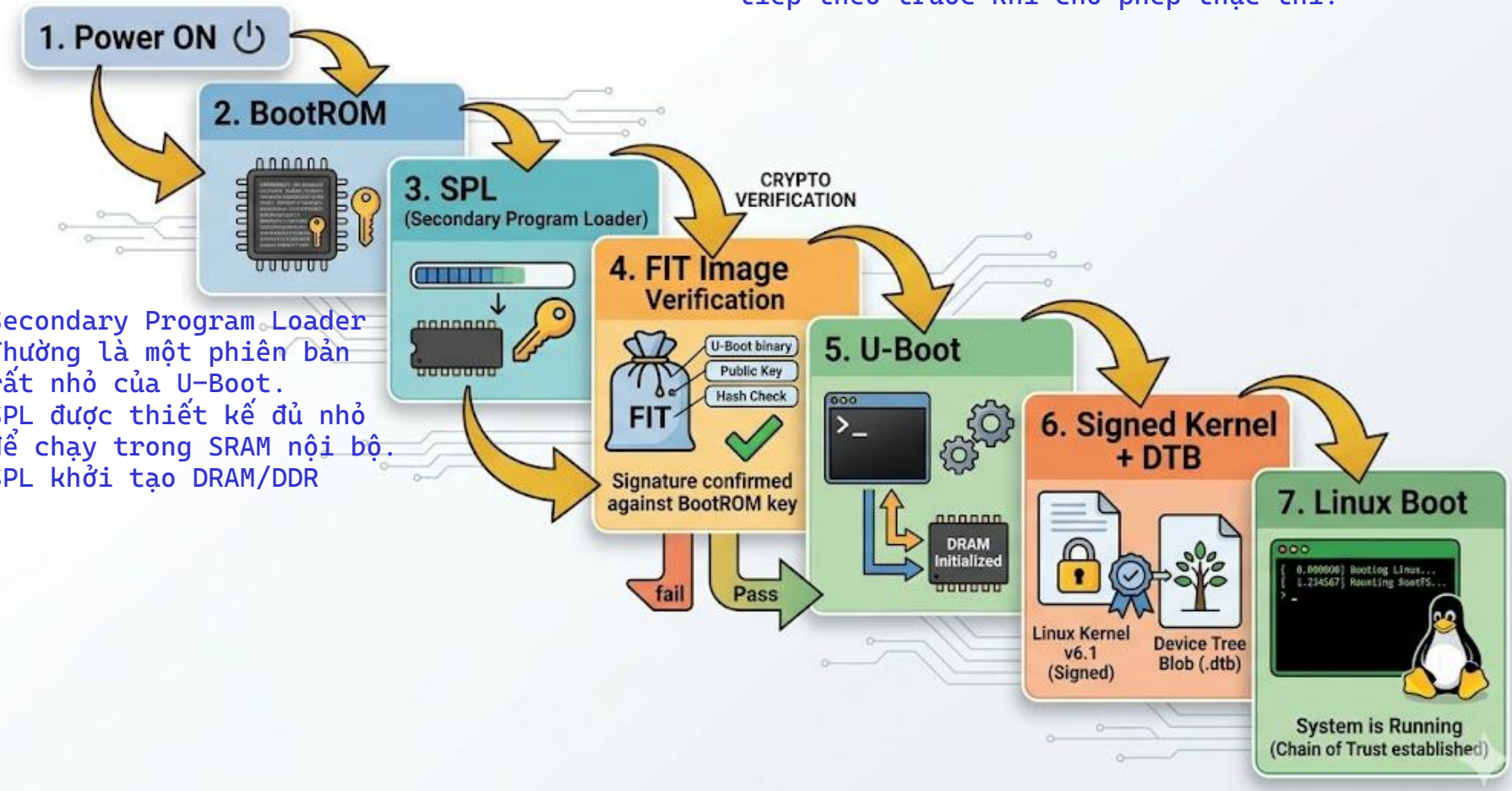
FPT Education

FPT UNIVERSITY

Example: NXP i.MX8 Secure Boot Flow

Power ON → BootROM → SPL
→ xác minh FIT → U-Boot
→ xác minh Kernel + DTB → Linux Boot

khác với Pi:
hệ thống không chỉ “nạp chương trình kế tiếp”,
mà mỗi tầng có thể xác minh tính hợp lệ của tầng
tiếp theo trước khi cho phép thực thi.



Secondary Program Loader
Thường là một phiên bản
rất nhỏ của U-Boot.
SPL được thiết kế đủ nhỏ
để chạy trong SRAM nội bộ.
SPL khởi tạo DRAM/DDR

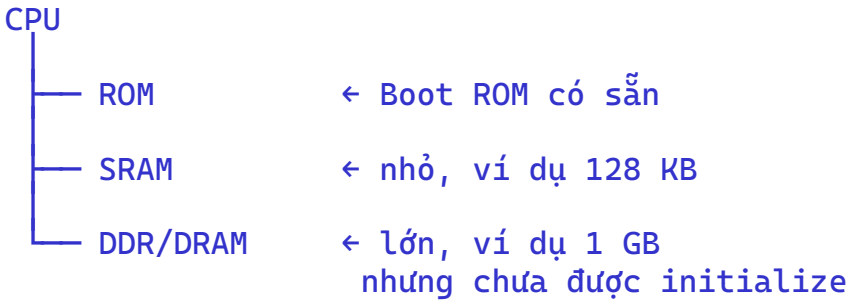
FPT Education
FPT UNIVERSITY

Bootloader Challenges

- Must initialize DRAM before stack
- No OS, no libc, bare-metal execution
- Often multi-stage: SPL → full loader

 Bootloader complexity comes from doing everything before the system exists

bootloader phải chuẩn bị hệ thống trong khi gần như chưa có bất kỳ dịch vụ hệ thống nào để dựa vào.



Das U-Boot

- De facto embedded bootloader
- Supports ARM, RISC-V, MIPS, PowerPC
- Interactive shell + scripting

 U-Boot is both bootloader and debugging tool during development

FPT Education

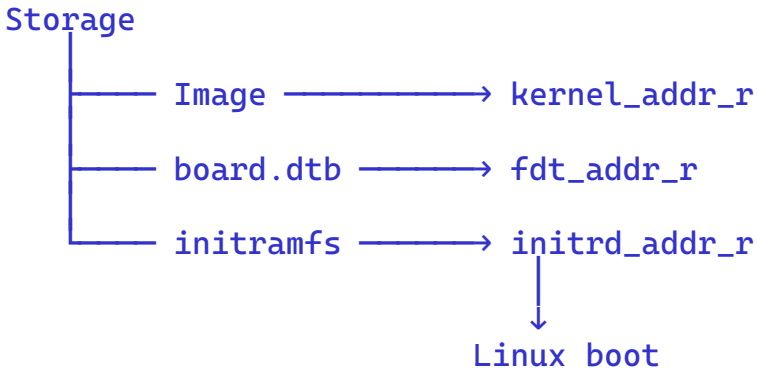
FPT UNIVERSITY

U-Boot Environment Variables

Boot dùng các biến này để quyết định:
boot cái gì, từ đâu, load vào đâu và truyền tham số gì cho kernel.

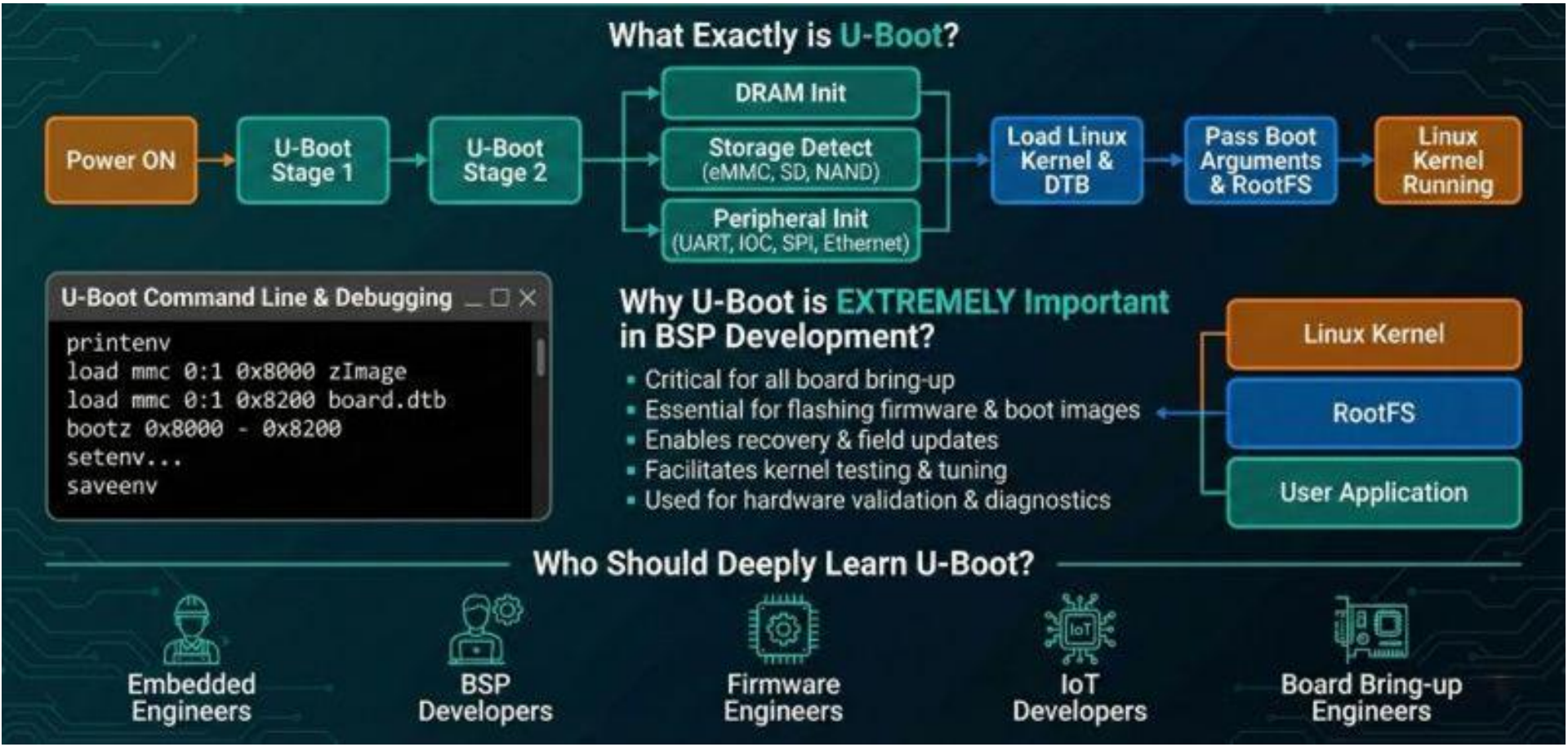
Variable		Purpose
	Sau khi hết bootdelay, U-Boot sẽ thực thi: run bootcmd	
bootcmd	bootcmd=run boot_linux setenv bootcmd 'load mmc 0:1 ${kernel_addr_r} Image; booti ${kernel_addr_r} - ${fdt_a	Command automatically executed during boot
bootargs		Linux kernel command line
	bootargs=console=ttyAMA0,115200 root=/dev/mmcblk0p2 rw	
bootdelay	bootdelay=3	Delay before autoboot
		Hit any key to stop autoboot: 3 nếu k bấm U-boot chạy bootcmd
	địa chỉ kernel được load vào RAM	
kernel_addr_r	kernel_addr_r=0x82000000	RAM address for kernel
	Flattened Device Tree	
fdt_addr_r	Flattened Device Tree	RAM address for DTB
		0x82000000 — Linux Kernel … 0x88000000 — Device Tree … 0x89000000 — initramfs
Dùng để lưu: initramfs hoặc initrd		
initrd_addr_r		RAM address for initramfs
boot_targets=mmc0 usb0 dhcp		
boot_targets		Boot device priority
		1. MMC/SD ↓ fail 2. USB ↓ fail 3. Network/DHCP
stdin/stdout/stderr	stdin=serial stdout=serial stderr=serial →	Keyboard/input → UART Output → UART Error → UART Console devices

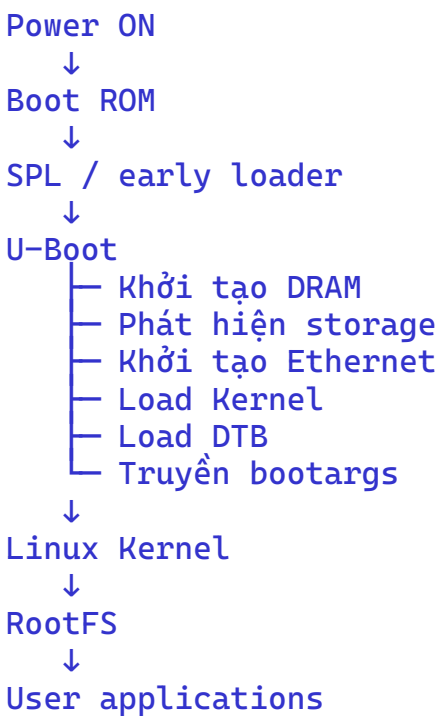
xác định thiết bị console:


Storage
Image
kernel_addr_r
board.dtb
fdt_addr_r
initramfs
initrd_addr_r
Linux boot

FPT Education
FPT UNIVERSITY

U-Boot: Brings Your Board to Life

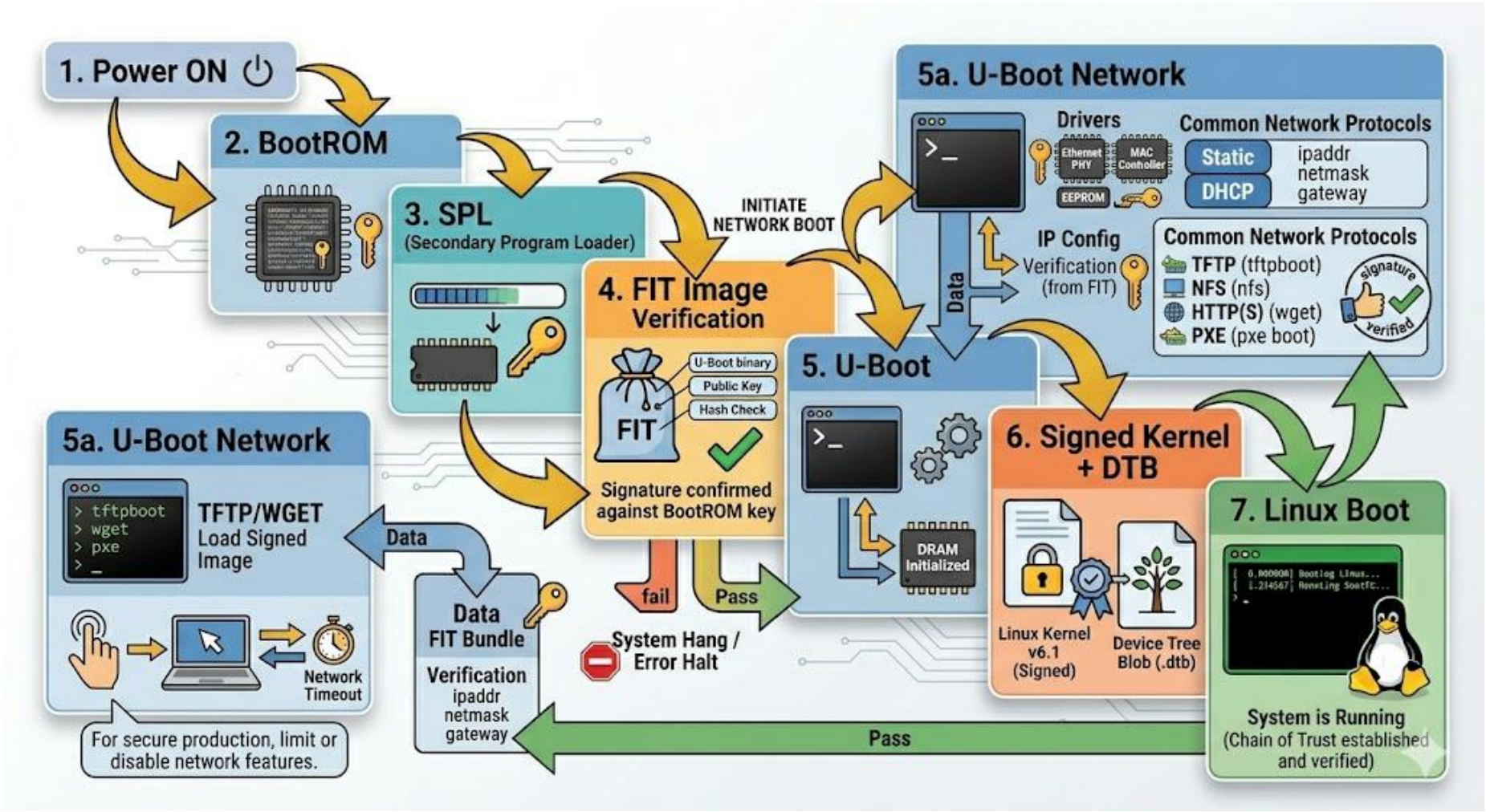



Power ON
↓
Boot ROM
↓
SPL / early loader
↓
U-Boot
├─ Khởi tạo DRAM
├─ Phát hiện storage
├─ Khởi tạo Ethernet
├─ Load Kernel
├─ Load DTB
└─ Truyền bootargs
↓
Linux Kernel
↓
RootFS
↓
User applications

FPT Education

FPT UNIVERSITY

U-Boot Network & Storage

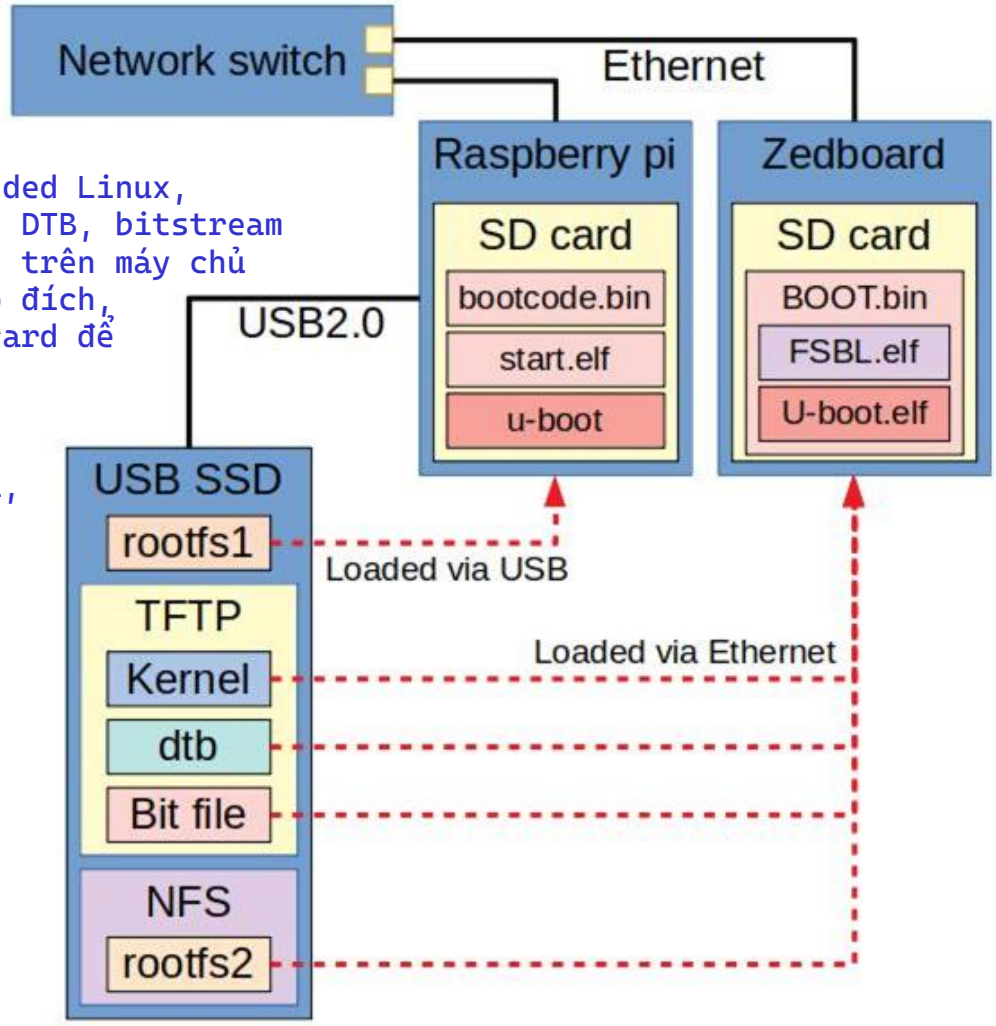


FPT Education
FPT UNIVERSITY

U-Boot Network

Khi phát triển embedded Linux, chỉ cần thay kernel, DTB, bitstream hoặc nội dung rootfs trên máy chủ rồi khởi động lại bo đích, giảm việc tháo SD card để chép lại file.

Trong hình, rootfs1 phục vụ Raspberry Pi, còn rootfs2 phục vụ ZedBoard.



Thành phần	Vai trò
Network switch	Kết nối Raspberry Pi và ZedBoard qua Ethernet
SD card trên Raspberry Pi	Chứa các thành phần khởi động: `bootcode.bin`, `start.elf`, U-Boot
USB SSD	Lưu hệ thống file của Pi và các file cung cấp cho ZedBoard
SD card trên ZedBoard	Chứa `BOOT.bin`, trong mô hình này đóng gói FSBL và U-Boot
TFTP	Cung cấp kernel, DTB và bitstream FPGA qua mạng
NFS	Cung cấp root filesystem để Linux trên ZedBoard sử dụng qua mạng

Raspberry Pi khởi động trước
Pi đọc các thành phần khởi động trên SD card, rồi khởi động Linux và sử dụng rootfs1 trên USB SSD.
Khi Linux trên Pi hoạt động, Pi chạy hai dịch vụ:
TFTP server: phục vụ các file dùng để khởi động ZedBoard.
NFS server: chia sẻ thư mục rootfs2 làm hệ thống file gốc cho ZedBoard.
Hình lược bỏ vị trí kernel của chính Raspberry Pi.

ZedBoard khởi động qua U-Boot
Quá trình diễn ra theo thứ tự:
Boot ROM đọc BOOT.bin từ SD card.
FSBL – First Stage Boot Loader khởi tạo phần cứng cần thiết, đặc biệt là RAM,
rồi chuyển quyền cho U-Boot.
U-Boot sử dụng Ethernet để tải file từ TFTP server trên Pi:
File Công dụng
Kernel Nhân Linux chạy trên ZedBoard
DTB Mô tả phần cứng cho kernel
Bit file Bitstream để cấu hình phần logic FPGA của Zynq
U-Boot có thể nạp bitstream vào FPGA, sau đó khởi chạy kernel
và truyền DTB cho kernel.
Linux trên ZedBoard mount rootfs2 qua NFS để truy cập /sbin/init, chương trình, thư viện và các file hệ thống.

Điểm cần phân biệt: U-Boot tải kernel và DTB vào RAM bằng TFTP; sau khi kernel chạy,
Linux truy cập root filesystem qua NFS. Rootfs không cần được tải toàn bộ vào RAM trước.

FPT Education
FPT UNIVERSITY

Device Tree (DTB)

- Hardware described in **.dts → compiled to .dtb**
- Passed from bootloader to kernel at boot time
- Eliminates hardcoded board-specific kernel code

 Device Tree enables one kernel binary for many hardware platforms

FPT Education
FPT UNIVERSITY

Device Tree Source (DTS)

- Human-readable hardware description
- Written in text format
- Compiled into DTB binary

Flow:

DTS → dtc → DTB → Linux Kernel

Extensions:

- `.dts` → board file
- `.dtsi` → shared include file
- `.dtb` → compiled binary

```
/ {                                   tên mô tả bo mạch
    model = "Custom Embedded
Board";
    Node RAM:
    memory@80000000 {đây là bộ nhớ RAM.
        device_type = "memory";
        reg = <0x80000000
0x40000000>; node RAM có địa chỉ bắt đầu
    };                                là 0x80000000.
                                      1 GiB RAM.
    Node UART:                        Node UART tại địa chỉ 0x10000000
    uart0: serial@10000000 {
        compatible =
"ns16550a";                           Vùng thanh ghi bắt đầu tại 0x10000000,
        reg = <0x10000000             kích thước 0x100 = 256 byte
0x100>;
Thông tin ngắt  interrupts = <5>;
        status = "okay";
    };                                Thiết bị được cho phép sử dụng
};
```

hai giá trị lần lượt là địa chỉ bắt đầu và kích thước.

Nhãn để nơi khác tham chiếu bằng &uart0

Cho biết thiết bị tương thích với UART NS16550A để kernel tìm driver phù hợp

Phần mô tả dùng chung, thường chứa cấu hình SoC để nhiều bo mạch sử dụng lại

FPT Education
FPT UNIVERSITY

Device Tree Compiler (dtc)

Purpose

- Converts DTS ↔ DTB
- Validates syntax
- Used during kernel build

Compile Device Tree

DTS → DTB

```
dtc -I dts -O dtb -o board.dtb board.dts
```

DTB → DTS

```
dtc -I dtb -O dts -o board.dts board.dtb
```

FPT Education
FPT UNIVERSITY

Viewing Device Tree in Linux

Current running DT

```
cat /proc/device-tree/model
```

Dump live DT

```
dtc -I fs -O dts
/sys/firmware/devicetree/base
```

Useful for:

- debugging drivers
- verifying overlays
- checking hardware config

Ví dụ: đã bật I²C nhưng thiết bị chưa hoạt động
→ xem node I²C tương ứng, kiểm tra thuộc tính status có là "okay" và cấu hình có đúng không.

FPT Education

FPT UNIVERSITY

Device Tree Overlays

Device Tree Overlay là một phần mô tả phần cứng bổ sung, dùng để sửa Device Tree gốc mà không cần biên dịch lại toàn bộ DTB. Ví dụ: bật SPI, thêm cảm biến I²C hoặc khai báo phần cứng mới trên FPGA.

- Small DT fragment
- Dynamically modifies base device tree
- Enables/disables peripherals without rebuilding full DTB

Very common on:

- Raspberry Pi 4
- BeagleBone capes
- FPGA platforms

```
/dts-v1/;
/plugin/;

/ {
    fragment@0 {
        target = <&spi0>;
        __overlay__ {
            status = "okay";
        };
    };
};
```

Khai báo phiên bản cú pháp Device Tree.

Cho biết đây là overlay, có thể tham chiếu các nhãn trong Device Tree gốc.

Một phần thay đổi; một overlay có thể chứa nhiều fragment.

Chọn node mang nhãn spi0 trong cây gốc để sửa.

Chứa các thuộc tính hoặc node cần bổ sung, cập nhật.

Đánh dấu bộ điều khiển SPI này được phép sử dụng.

```
trước overlay:
/* Trong Device Tree gốc */
spi0: spi@... {
    status = "disabled";
};

Sau khi áp dụng overlay:
status = "okay";
```

Kernel có thể khởi tạo bộ điều khiển SPI nếu có driver và các cấu hình cần thiết. Bật bộ điều khiển SPI chưa đồng nghĩa đã khai báo thiết bị SPI kết nối với nó; cảm biến hoặc màn hình thường cần node con riêng.

Overlay thường gặp trên Raspberry Pi, BeagleBone với bo mở rộng cape và nền tảng FPGA, vì các hệ thống này hay thay đổi phần cứng ngoại vi.

Why DT Overlays Matter

Advantages:

- Modular hardware support
- Easier prototyping
- Dynamic peripheral enablement
- No full kernel rebuild
- Useful for HATs/capes/modules

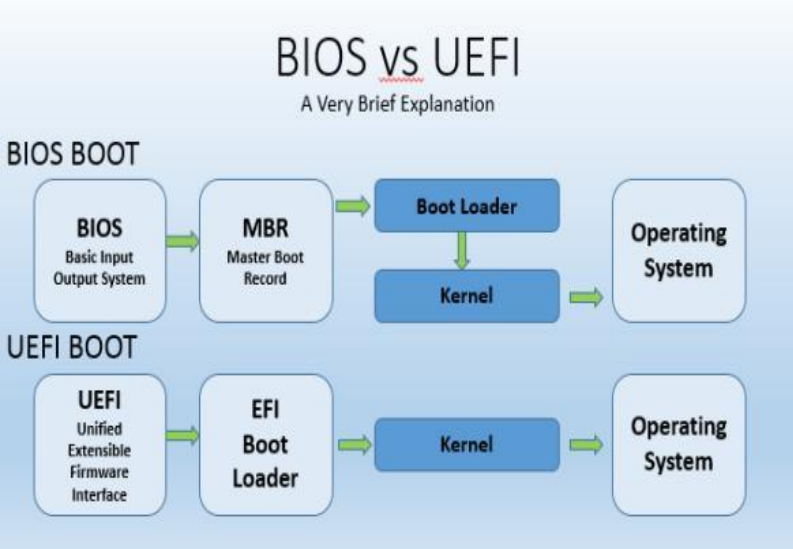
Used heavily in:

- robotics
- industrial Linux
- FPGA systems
- rapid prototyping

FPT Education
FPT UNIVERSITY

Other Bootloaders

- GRUB for PC systems
- UEFI replaces legacy BIOS
- coreboot for fast boot



Embedded → U-Boot | PC → UEFI + GRUB

Porting U-Boot to a New Board

Adapting U-Boot to initialize and boot a custom embedded board

Task	Purpose
DDR initialization	Enable external RAM
UART setup	Early serial debugging
Clock configuration	Configure PLLs/clocks
Pin multiplexing	Configure SoC pins
Boot media support	SD/eMMC/NAND/NOR
Ethernet support	TFTP/NFS boot
Device Tree support	Describe board hardware

Summary

- Bootloaders initialize hardware
- U-Boot dominates embedded Linux
- Device Tree decouples hardware
- FIT image supports secure boot